

OMI Rule-Set Type Engine

Core Claim

The Haskell engine should not begin with runtime behavior, UI, JSON, networking, or canvas rendering.

It should begin with **rules**.

A rule is the smallest declared relation that can be judged.

A rule set is an arranged family of rules.

A decision table is a rule set made inspectable.

A circular slide array is a decision table made orientable.

A nomogram pleth is the selector that chooses which relation surface is being read.

The gauge is the selected addressing frame.

The observer reads the framed result.

The receipt accepts it.

```
Rule
  ↓
Rule Set
  ↓
Decision Table
  ↓
Circular Slide Array
  ↓
Nomogram / Pleth Selection
  ↓
Gauge Addressing
  ↓
Observer
  ↓
Receipt
```

Haskell Target

The Haskell project should become a pure type engine for this chain.

Not:

```
Canvas app first
```

But:

```
Rule engine first  
Canvas projection later
```

The type engine should define:

```
Rule  
RuleSet  
DecisionTable  
SlideArray  
Pleth  
Gauge  
Observer  
Receipt
```

without giving them external authority.

They are pure OMI relations.

Rule

A rule is a local truth condition.

```
(rule . relation)
```

It has:

```
condition  
operator  
result  
scope
```

In OMI-Lisp form:

```
(rule
  (condition . C1)
  (operator . F6)
  (result . candidate))
```

Rule Set

A rule set is a named arrangement of rules.

```
(rule-set
  (name . constitutional-grid)
  (rules . (...)))
```

The rule set does not execute by itself.

It organizes possible judgments.

Decision Table

A decision table makes the rule set explicit.

Each cell contains:

```
question
algorithm
citation slot
proof form
operator code
result
```

This gives the expanded constitutional grid:

```
8 questions
x
7 algorithms
x
4 citation slots
x
2 proof forms
```

```
=  
448 cells
```

Each cell is labeled by a Wittgenstein operator code:

```
F0..FE = operator rows  
FF      = tautology / default closure
```

Wittgenstein Operator Field

The F* gauge family maps directly to the 16 truth-function rows.

```
F0 = contradiction  
F1 = NOR  
F2 = converse nonimplication  
F3 = not p  
F4 = material nonimplication  
F5 = not q  
F6 = XOR  
F7 = NAND  
F8 = AND  
F9 = XNOR  
FA = q  
FB =  $p \rightarrow q$   
FC = p  
FD =  $p \leftarrow q$   
FE = OR  
FF = tautology / canonical closure
```

The low nibble selects the operator.

The high nibble marks the gauge family.

```
F* = gauge row  
*   = operator / dialect / orbit offset
```

Circular Slide Array

The decision table should be readable as a circular slide array.

The slide array lets the same rule set be rotated, aligned, and compared without changing the underlying rules.

This is where the circular slide rule model enters.

The rule set remains fixed.

The gauge selects the reading.

Nomogram / Pleth

The nomogram pleth is the alignment line.

It does not create truth.

It selects which surface is read.

For squares:

$$a^2 - b^2 = (a - b)(a + b)$$

This gives a two-factor split:

difference
span

For cubes:

$$a^3 - b^3 = (a - b)(a^2 + ab + b^2)$$

This gives:

line
surface
volume

Together:

2 square factors
×
3 cubic factors
=
6 gauge color lanes

Six Color Factors

JSON Canvas gives six preset colors.

OMI can use them as six factor lanes:

```
1 red      = contradiction / reject
2 orange   = difference / falsification
3 yellow   = transition / candidate
4 green    = constructive / accepted
5 cyan     = projection / witness
6 purple   = tautology / canonical closure
```

The colors are not decoration.

They are factor lanes.

They encode how the rule is being read.

Tetragrammatron and Metatron

The observer split becomes:

```
Tetragrammatron
= four-way address / gate observer

Metatron
= rule-set / receipt / closure observer
```

Tetragrammatron reads the operational frame:

```
o---o / --- / ?---? @---@
```

Metatron reads whether the rule set closes lawfully into receipt.

Haskell Type Engine Shape

The pure Haskell type target should look conceptually like:

```
data Rule
data RuleSet
```

```
data DecisionTable
data Operator
data CitationSlot
data ProofForm
data SlideArray
data Pleth
data Gauge
data Observer
data Receipt
```

But each of these must reduce to OMI relations.

No type should become magical.

The rule is:

```
If it cannot reduce to a relation,
it does not enter the kernel.
```

Canon Sentence

Rules define local truth conditions.

Rule sets arrange them into decision tables.

Decision tables become circular slide arrays.

The nomogram pleth selects the addressing frame.

The gauge orients the frame.

The observer reads the result.

The receipt accepts the closure.

The six color lanes arise from the 2×3 factorization of square difference and cubic surface laws.